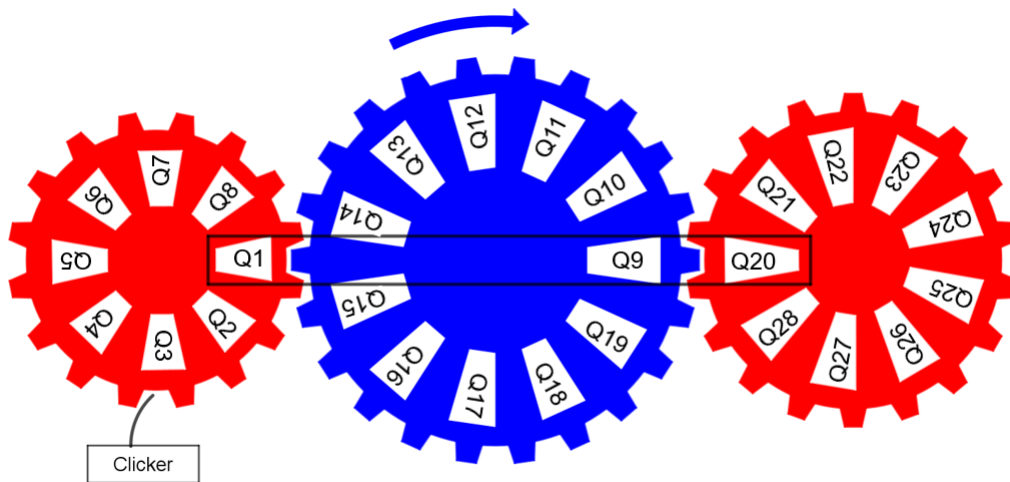


Unlocking S3



To unlock the final question, first enter all the clues you have received in your cogs diagram (see Qn 20).

You should find that, with the three numbers concatenated, a 9-digit Fibonacci number appears in the rectangle. All other 9-digit Fibonacci numbers will appear if you rotate the cogs a sufficient number of times.

Rotating the blue cog (it only moves clockwise) causes the two red cogs to move anticlockwise. **Notice how the clicker records movement.**

If you were to rotate the blue cog so that the next three numbers appear in the rectangle and you treated these as lengths of the sides of a triangle, you should find that this triangle is acute-angled.

To unlock the final question, you will need to decide which of the Fibonacci numbers is the best one to start from and rotate the cogs so this number appears in the rectangle. Then you need to find the minimum value of $p \times q \times r \times s$ where, starting from a Fibonacci number:

p clicks takes you to three numbers that form a Primitive Pythagorean Triple;

a further q clicks takes you to three numbers that are the sides of a triangle containing a 60° angle;

a further r clicks takes you to three numbers that are the sides of a triangle with integer area;

a further s clicks takes you to a 9-digit Fibonacci number.

On the Ritangle login page, enter the minimum possible product $p \times q \times r \times s$ to unlock the final question.

Working

```
In [1]: from math import acos, degrees, sqrt, prod, gcd

global clues
clues = {
    "Q01" : 267,
    "Q02" : 851,
    "Q03" : 259,
    "Q04" : 433,
    "Q05" : 493,
    "Q06" : 165,
    "Q07" : 701,
    "Q08" : 102,
    "Q09" : 914,
    "Q10" : 494,
    "Q11" : 468,
    "Q12" : 460,
    "Q13" : 143,
    "Q14" : 150,
    "Q15" : 832,
    "Q16" : 580,
    "Q17" : 299,
    "Q18" : 334,
    "Q19" : 408,
    "Q20" : 296,
    "Q21" : 763,
    "Q22" : 155,
    "Q23" : 145,
    "Q24" : 168,
    "Q25" : 437,
    "Q26" : 733,
    "Q27" : 154,
    "Q28" : 141,
}

global nineDigitFibonacci
nineDigitFibonacci = [ # https://planetmath.org/Listoffibonacci-numbers
    [102, 334, 155],
    [165, 580, 141],
    [267, 914, 296],
    [433, 494, 437],
    [701, 408, 733]
]
```

```

def rotateList(l,n=1): # +n will always rotate the cog in the right direction
    return l[n:] + l[:n]

def rotateUntil(l,value):
    count = 0
    while l[0] != value:
        l = rotateList(l)
        count +=1
        if count >= len(l):
            raise KeyError
    return l

def rotateAll(lists,n=1):
    newList = []
    for list in lists:
        newList.append(rotateList(list,n))
    return newList

def isPrimitivePythagoreanTriple(sides):
    sides.sort()
    a,b,c = sides
    return a**2 + b**2 == c**2 and gcd(a,b,c) == 1

def calculateAngle(opposite,sideA,sideB): # Returns in Degrees, uses cos rule
    return degrees( acos( (sideA**2 + sideB**2 - opposite**2) / (2*sideA*sideB) ) )

def isValidTriangle(sides):
    sides.sort()
    return sides[0] + sides[1] > sides[2]

def isContainsSixtyDegrees(sides):
    if not isValidTriangle(sides):
        return False
    angles = []
    for oppositeSide in sides:
        otherSides = sides.copy()
        otherSides.remove(oppositeSide)
        angles.append(calculateAngle(oppositeSide,*otherSides))

    for angle in angles:
        mod = angle % 60
        if min(60-mod,mod) <= 0.000000000001: # Accounts for floating point error
            return True
    return False

def isIntArea(sides): # https://en.wikipedia.org/wiki/Heron%27s_formula
    if not isValidTriangle(sides):
        return False

    a,b,c = sides
    s = 0.5 * sum(sides)
    area = sqrt(s*(s-a)*(s-b)*(s-c))
    mod = area % 1
    if min(1-mod,mod) <= 0.000000000001: # Accounts for floating point error
        return True
    else:
        return False

def isFibonacciNumber(sides):
    return any(number == sides for number in nineDigitFibonacci)

def clicksUntilConditionMet(cogs,function):
    i = 0
    functionIsTrue = False
    while not functionIsTrue:
        i += 1
        cogs = rotateAll(cogs)
        if function([cog[0] for cog in cogs]):
            functionIsTrue = True
        if i > 792: # Raises error if the condition is never met
            raise KeyError
    return [i*2,cogs] # Multiply by two because of the way the clicker counts.

cog0 = [clues["Q01"],clues["Q02"],clues["Q03"],clues["Q04"],clues["Q05"],clues["Q06"],clues["Q07"],clues["Q08"]]
cog1 = [clues["Q09"],clues["Q10"],clues["Q11"],clues["Q12"],clues["Q13"],clues["Q14"],clues["Q15"],clues["Q16"],clues["Q17"],clues["Q18"],clues["Q19"]]
cog2 = [clues["Q20"],clues["Q21"],clues["Q22"],clues["Q23"],clues["Q24"],clues["Q25"],clues["Q26"],clues["Q27"],clues["Q28"]]

products = []

for fibonacci in nineDigitFibonacci:

    cog0 = rotateUntil(cog0, fibonacci[0])
    cog1 = rotateUntil(cog1, fibonacci[1])
    cog2 = rotateUntil(cog2, fibonacci[2])

    cogs = [cog0,cog1,cog2]

    p,cogs = clicksUntilConditionMet(cogs,isPrimitivePythagoreanTriple)
    q,cogs = clicksUntilConditionMet(cogs,isContainsSixtyDegrees)
    r,cogs = clicksUntilConditionMet(cogs,isIntArea)
    s,cogs = clicksUntilConditionMet(cogs,isFibonacciNumber)

    products.append(prod([p,q,r,s]))

products.sort()
print(f"Minimum Product: {products[0]}")

```

Minimum Product: 14152320